

Day 1 Solution: Add 2 Integers

6 Python Ways to Add Two Numbers!!

Here's a collection of Python solutions to add two integers — from the classic + to creative ways like `sum()`, `operator.add`, and even bitwise addition without using + at all.

Fun to learn, interview-ready, and shows Python's flexibility!

Intuition

When asked to return the sum of two integers, the obvious answer is:

```
return num1 + num2
```

But did you know Python offers several fun and interesting alternatives?

Let's look at them!

1. The classic way (best in practice)

```
class Solution:
```

```
    def sum(self, num1: int, num2: int) -> int:
```

```
        return num1 + num2
```

Clean, fast, and exactly what's needed.

Use this in production code.

2. Using the built-in sum()

```
class Solution:
```

```
def sum(self, num1: int, num2: int) -> int:  
    return sum([num1, num2])
```

This adds all numbers in an iterable — here, our list of two numbers.

3. Using operator.add

```
import operator  
  
class Solution:  
    def sum(self, num1: int, num2: int) -> int:  
        return operator.add(num1, num2)
```

The same as `num1 + num2`, but as a function — sometimes useful in higher-order functions or functional programming.

4. Lambda function

```
class Solution:  
    def sum(self, num1: int, num2: int) -> int:  
        add = lambda x, y: x + y  
        return add(num1, num2)
```

Mostly just for fun, but shows you can define your own inline function.

5. Using functools.reduce

```
from functools import reduce  
  
class Solution:  
    def sum(self, num1: int, num2: int) -> int:  
        return reduce(lambda x, y: x + y, [num1, num2])
```

Not practical here, but if you had a longer list, reduce can fold it into a single value.

6. Bitwise addition (without +)

For interview fun or if you're challenged not to use + at all:

class Solution:

```
def sum(self, num1: int, num2: int) -> int:
```

```
    MAX = 0x7FFFFFFF
```

```
    mask = 0xFFFFFFFF
```

```
    while num2 != 0:
```

```
        num1, num2 = (num1 ^ num2) & mask, ((num1 & num2) << 1) & mask
```

```
    return num1 if num1 <= MAX else ~(num1 ^ mask)
```

This simulates addition by summing without carry (^) and then adding the carry (& shifted left).

Complexity

- Time complexity:
O(1) — adding two integers takes constant time.
- Space complexity:
O(1) — only basic variables used.

Summary

Best in practice: num1 + num2

Fun to know: sum(), operator.add, lambda, reduce

Advanced: bitwise addition without + (for interview puzzles)

Hope this helps!